

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What happens when this code runs?

```
def first():
    second()
```

```
def second():
    third()
```

```
def third():
    raise RuntimeError("Failure")
```

```
try:
    first()
except RuntimeError:
    print("Handled")
```

- A)Handled
- B)TypeError
- C)NameError
- D)RuntimeError cannot propagate through functions

Q2. What is the output?

```
try:
    raise ValueError
except ValueError:
    print("A")
    raise
```

- A)B
- B)A, and execution ends normally
- C)No Output
- D)A, followed by propagation of the same exception

Q3. What will be printed?

```
def fun():
    try:
        return 10
    finally:
        print("Finally")
```

```
print(fun())
```

- A)10 only
 - B)10 followed by Finally
 - C)Finally only
 - D)Finally followed by 10
- Q4. What is the output?**

```
def outer():
    x = 10

    def inner():
        return x + 5
```

```
    return inner
```

```
f = outer()
print(f())
```

- A)NameError
- B)None
- C)15
- D)10

Q5. What is the output?

```
try:
    x = int("abc")
except ValueError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

- A)B C
- B)B
- C)A
- D)A C

Q6. Why is

```
try:
    print(10 / 0)
except Exception:
    print("A")
except ZeroDivisionError:
    print("B")
codes's second handler never
```



reached?

- A) ZeroDivisionError occurs before Exception
- B) Exception is a superclass of ZeroDivisionError
- C) except blocks execute randomly
- D) ZeroDivisionError is not an exception

Q7. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
            numbers)
    )
)
```

```
print(result)
```

- A) [2, 4, 6, 8, 10]
- B) [2, 4]
- C) [4, 8]
- D) [1, 3, 5]

Q8. What will be printed?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, b=20))
```

- A) TypeError
- B) SyntaxError
- C) 20
- D) 30

Q9. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the

same Python process, what is normally printed?

- A) Module loaded
- Module loaded
- 10
- B) 10
- 10
- C) ImportError
- D) Module loaded
- 10

Q10. What will be the output?

```
x = 10
```

```
def outer():
    x = 20
```

```
    def inner():
        global x
        x = 30
```

```
    inner()
    print(x)
```

```
outer()
print(x)
```

- A) 20
- 10
- B) 30
- 30
- C) 30
- 20
- D) 20
- 30

Q11. What is the output?

```
def fun(a=10, b=20):
    return a - b
```

```
print(fun(30))
```

- A) 10
- B) 30
- C) -10
- D) 20

Q12. What is the output?

```
def fun(x):
    x += 10
```



```
return x
```

```
x = 5  
y = fun(x)
```

```
print(x, y)
```

A)15 15

B)15 5

C)5 5

D)5 15

Q13. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):  
    if age < 18:  
        raise InvalidAgeError("Age is  
too low")  
    return "Allowed"
```

```
try:  
    print(check_age(15))  
except InvalidAgeError as e:  
    print(type(e).__name__)  
    print(e)
```

A)Nothing, because custom exceptions cannot be caught

B)InvalidAgeError

Age is too low

C)InvalidAgeError

15

D)Exception

Age is too low

Q14. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

```
print(fun(3))
```

A)RecursionError

B)6

C)3

D)TypeError

Q15. What is the output?

```
def fun(x):  
    return x * 2
```

```
a = fun
```

```
print(a(5))
```

A)5

B>Error

C)10

D)func(5)

Q16. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

```
variable = 10
```

```
fun()
```

A)UnboundLocalError

B)50

C)NameError

D)10

Q17. What will happen?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal y  
        y = 20
```

```
    inner()
```

```
outer()
```

A)20

B)NameError

C)10

D)SyntaxError

Q18. What happens here?

```
def fun(a):  
    return a
```

```
print(fun())
```

A)0

B)TypeError



C)NameError

D)None

Q19. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()
```

A)Only Hello

B)10

Hello

C)Error

D)x

display

Q20. What will be printed?

```
try:
```

```
    print(10 / 0)
```

```
except ZeroDivisionError:
```

```
    print("A")
```

```
except Exception:
```

```
    print("B")
```

A)A

B)Nothing

C)A B

D)B

Q21. What is the output?

```
def fun(x):
```

```
    if x == 0:
```

```
        return
```

```
    return x + fun(x - 1)
```

```
print(fun(3))
```

A)None

B)6

C)5

D)3

Q22. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

A)NameError

B)10

C)None

D)20

Q23. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

A)15

B>Error

C)5

D)10

Q24. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():
```

```
    raise MyError("Something went  
wrong")
```

```
try:
```

```
    test()
```

```
except MyError as e:
```

```
    print(type(e).__name__)
```

A)MyError

B)Something went wrong

C)Exception

D)TypeError

Q25. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

A)Python automatically renames the local file

B)random cannot be imported

C)The local file can shadow the intended standard-library module

D)Python will always reject it



Q26. Which object keeps track of modules that have already been imported during the current Python process?

- A) `__module__`
- B) `sys.imports`
- C) `sys.modules`
- D) `sys.path`

Q27. What will be printed?

```
def make_counter():  
    count = 0  
  
    def counter():  
        nonlocal count  
        count += 1  
        return count  
  
    return counter
```

```
a = make_counter()  
b = make_counter()
```

```
print(a())  
print(a())  
print(b())  
print(a())
```

A) 1

2

1

3

B) 1

2

3

4

C) 0

1

0

2

D) 1

1

1

1

Q28. What is the output?

```
def fun():  
    try:  
        return 10
```

finally:
return 20

print(fun())

A) None

B) 10

C) 20

D) RuntimeError

Q29. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:  
            raise ValueError("negative")  
        return x  
    except ValueError:  
        return 100  
    finally:  
        print("cleanup")
```

print(fun(-5))

A) cleanup

-5

B) cleanup

100

C) negative

cleanup

D) 100

cleanup

Q30. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A) B

B) A

C) Nothing

D) A and B

Q31. What is the output?

```
try:
```



```

    print(10 / 0)
except Exception:
    print("A")
except ZeroDivisionError:
    print("B")

```

A)SyntaxError

B)B

C)A

D)A B

Q32. What happens when this function is called?

```

def fun(a=10, b):
    return a + b

```

A)NameError

B)TypeError

C)It executes correctly

D)SyntaxError

Q33. What happens?

```

def fun(a, b):
    return a + b

```

```

print(fun(10, a=20))

```

A)30

B)TypeError

C)SyntaxError

D)20

Q34. What will happen?

```

def fun(a, b=10):
    return a + b

```

```

print(fun(b=5, a=20))

```

A)SyntaxError

B)25

C)TypeError

D)15

Q35. What happens here?

```

from math import sqrt

```

```

print(sqrt(36))

```

A)NameError

B)36

C)6.0

D)6

Q36. What does the following function return?

```

def fun():
    print("Hello")

```

```

x = fun()

```

```

print(x)

```

A)None

B)Hello

C)Hello None

D>Error

Q37. What is the output?

```

def fun():
    try:
        print("A")
        raise ValueError
    except ValueError:
        print("B")
        return 10
    else:
        print("C")
        return 20
    finally:
        print("D")

```

```

print(fun())

```

A)A

B

C

D

10

B)A

B

D

10

C)A

B

10

D

D)A

C

D

20



Q38. What will normally happen?

```
import math
```

```
import math
```

A)SyntaxError

B)The module is normally loaded once and reused

C)The module is normally loaded and executed twice

D)ImportError

Q39. if test.py has

```
print("A")
```

```
if __name__ == "__main__":
```

```
    print("B")
```

If test.py is executed directly, what will be printed?

A)B

B)Nothing

C)A

D)A followed by B

Q40. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

A)First imports only sqrt

B)Both import only sqrt

C)There is no difference

D)First provides the module namespace; second imports sqrt directly

Q41. What will be the output?

```
x = 10
```

```
def fun():
```

```
    print(x)
```

```
    x = 20
```

```
fun()
```

A)UnboundLocalError

B)20

C)10

D)NameError

Q42. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal x
```

```
        x = x + 5
```

```
    return x
```

```
inner()
```

```
return x
```

```
print(outer())
```

A)10

B)UnboundLocalError

C)15

D)5

Q43. What will be printed?

```
def fun():
```

```
    print("A")
```

```
    return
```

```
    print("B")
```

```
fun()
```

A)B

B)A

C)A B

D)Nothing

Q44. What will be printed?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        print(x)
```

```
    x = 30
```

```
    inner()
```

```
outer()
```

A)20

B)30

C)10

D)NameError



Q45. What happens here?

try:

```
    raise ValueError("Wrong value")
except ValueError as e:
    print(e)
```

A)ValueError

B)Nothing

C)Wrong Value

D)e

Q46. What is the purpose of this statement?

```
if __name__ == "__main__":
```

A)It creates the main function

B)It identifies code intended to run when the file is executed directly

C)It prevents syntax errors

D)It prevents all imports

Q47. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(
    lambda a, b: a * b,
    numbers
)
```

```
print(result)
```

A)10

B)24

C)9

D)0

Q48. What will be printed?

```
def fun(x=[]):
    x.append(10)
    return x
```

```
a = fun()
```

```
b = fun()
```

```
print(a)
```

```
print(b)
```

A)[10, 10]

[10]

B)[10]

[10, 10]

C)Error

D)[10]

[10]

Q49. What will be printed?

try:

```
    x = int("10")
```

except ValueError:

```
    print("A")
```

else:

```
    print("B")
```

finally:

```
    print("C")
```

A)A

B)B C

C)A C

D)B

Q50. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A)sqrt(25)

B)5.0

C)5

D)25

